Contrato de datos Firebase – EcoSmart Grid

Oct 13, 2026 · @Voltec

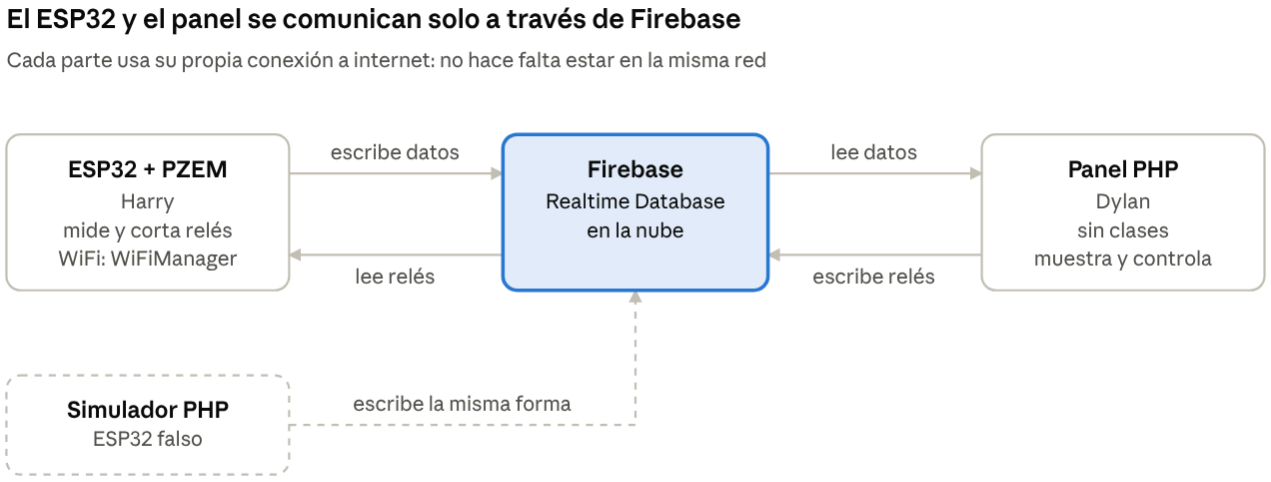
Objetivo

El ESP32 y el panel PHP comparten una sola base: Firebase Realtime Database. Este contrato fija qué ramas existen, qué datos tienen, de qué tipo son y quién escribe y lee cada una. Si todos respetan esta forma, cada parte puede avanzar por separado y después se conecta sin cambios.

Lo que pidieron los profes de PDI y dónde se resuelve en este documento:

Pedido de los profes	Dónde se resuelve
Que las dos partes usen Firebase	Toda la estructura de la base
Que el PHP pida los datos a Firebase	Quién escribe y quién lee
Que el usuario sepa si el ESP32 está conectado	Reglas: estado en línea
Cómo se conectan en la misma red	Arquitectura
Trabajar con datos falsos mientras Harry avanza	Datos simulados
Definir tablas, tipos y qué se manda y recibe	Estructura de la base
Sacar la programación orientada a objetos	Reparto del trabajo

Arquitectura



arquitectura · ESP32, Firebase y panel PHP

Hoy el ESP32 le habla directo a una PC en `192.168.100.136`, por eso todo tiene que estar en la misma red. Con Firebase cada parte se conecta a la nube por su cuenta: el ESP32 solo necesita internet, y eso ya lo resuelve WiFiManager con la red "Energhost-Config". El simulador ocupa el lugar del ESP32 mientras el hardware no está listo.

Estructura de la Realtime Database

La base es un árbol JSON con dos ramas principales: `dispositivos` guarda el estado actual de cada equipo y `historial` guarda una lectura por minuto. Se elige Realtime Database y no Firestore porque es la de tiempo real que pidieron los profes, es la mejor soportada en ESP32 y ya hay una prueba hecha en PHP.

Versión 2: cada usuario tiene su cuenta y sus equipos, cada equipo tiene 3 tomas con un relé por toma (un artefacto en cada toma), y cada equipo se identifica solo con su MAC. Ya no hay un dispositivo fijo `EcoSmart_01`.

Hay un solo sensor PZEM-004T por equipo: `lectura_actual` e `historial` guardan el total de las 3 tomas, no el consumo de cada artefacto.

```
usuarios/
  <uid>/                      nombre, email, password_hash, rol, creado
    dispositivos/<id>: true   equipos vinculados a la cuenta
usuarios_por_email/
  <sha1 del email>: <uid>     índice para buscar por email
dispositivos/
  EcoSmart_A1B2C3/
    info/                     mac, cantidad_reles (3), ultimo_arranque ·
duenio, nombre, vinculado
    vinculacion/codigo        "483920"
    tomas/                    toma_1: "Heladera", toma_2: "Microondas",
toma_3: "Pava eléctrica"
    lectura_actual/           voltaje, corriente, potencia, energia_kwh,
consumo_fantasma, timestamp   (total de las 3 tomas)
    reles/                    rele_1, rele_2, rele_3   (rele_N controla la
toma N)
    estado/                   ultima_conexion, ip, sobreconsumo, error_sensor
historial/
  EcoSmart_A1B2C3/<clave>/    potencia, energia_kwh, timestamp
eventos/
  EcoSmart_A1B2C3/<clave>/    usuario, nombre_usuario, accion, detalle,
timestamp
```

Campos nuevos o cambiados:

Ruta	Campo	Tipo	Valores
`usuarios/<uid>`	`rol`	texto	`usuario` o `admin`
`usuarios/<uid>`	`password_hash`	texto	hash de `password_hash()`, nunca la contraseña
`info`	`duenio`	texto	uid del usuario; no existe si el equipo está libre
`info`	`nombre`	texto	lugar del equipo (ej: "Cocina"), hasta 40 caracteres
`info`	`cantidad_reles`	número entero	3 (un relé por toma)
`info`	`mac`	texto	MAC del ESP32
`vinculacion`	`codigo`	texto	6 cifras, lo genera el ESP32 una sola vez

Ruta	Campo	Tipo	Valores
`tomas`	`toma_1, toma_2, toma_3`	texto	artefacto enchufado en cada toma, hasta 40 caracteres; sin nombre se muestra "Toma N"
`reles`	`rele_1`, rele_2, rele_3	número entero	0 = apagado, 1 = encendido; rele_N controla la toma N (GPIO18, 19 y 21)
`eventos/<id>`	`accion`	texto	`vinculado`, `desvinculado`, `encendido`, `apagado`

Las ramas `lectura_actual`, `estado` e `historial` no cambian respecto de la versión 1.

Quién escribe y quién lee

El ESP32 solo toca las ramas de su propio equipo; los usuarios y la actividad los maneja únicamente el panel PHP.

Rama	Escribe	Lee	Cuándo
`dispositivos/<id>/info` (mac, cantidad_reles)	ESP32	Panel	al arrancar
`dispositivos/<id>/info` (duenio, nombre)	Panel	Panel	al vincular, renombrar o quitar
`dispositivos/<id>/vinculacion/codigo`	ESP32	Panel	una vez, al primer arranque
`dispositivos/<id>/tomas`	Panel	Panel	al vincular o renombrar; se borra al quitar
`dispositivos/<id>/lectura_actual`	ESP32	Panel	cada 2 s
`dispositivos/<id>/estado`	ESP32	Panel	cada 10 s
`dispositivos/<id>/reles`	Panel (y ESP32 al cortar)	ESP32	al tocar el interruptor
`historial/<id>`	ESP32	Panel	cada 1 min
`usuarios`, `usuarios_por_email`	Panel	Panel	al registrarse o cambiar el rol

Rama	Escribe	Lee	Cuándo
`eventos/<id>`	Panel	Panel	en cada vínculo, encendido o apagado

Identidad y vinculación de equipos

Cada equipo se identifica solo y queda libre hasta que un usuario lo reclama con el código de su etiqueta.

1. **ID por MAC.** El firmware toma los últimos 6 caracteres de la MAC del ESP32 y arma el ID, por ejemplo `EcoSmart_A1B2C3`. Es único y nunca cambia.
2. **Código.** En el primer arranque sortea un código de 6 cifras y lo guarda en la memoria del ESP32 (`Preferences`), así no cambia aunque se corte la luz.
3. **Presentación.** Al conectarse escribe `info` (mac, cantidad_reles) y `vinculacion/codigo`, sin dueño. Muestra el ID, el código y el link del QR en el monitor serie; esos datos van a la etiqueta del gabinete.
4. **Reclamo.** El usuario entra a `/vincular` y escribe el ID, el código, el nombre del equipo y, si quiere, el artefacto de cada toma, o escanea el QR, que trae `?equipo=A1B2C3&codigo=483920`.
5. **Controles del panel**, en este orden:

Control	Si falla
El equipo existe en Firebase	"No encontramos ese equipo"
No tiene dueño	"Ese equipo ya está vinculado a otra cuenta"
El código coincide	"El código de vinculación es incorrecto"
Menos de 5 intentos fallidos	bloqueo de 10 minutos

6. **Asignación.** Se guardan `info/duenio`, `info/nombre`, los nombres de las tomas, la marca en `usuarios/<uid>/dispositivos` y el evento "vinculado".
7. **Quitar.** El dueño puede desvincular el equipo; queda libre y se vuelve a vincular con el mismo código.

Reglas de funcionamiento

Estado en línea. Si `estado/ultima_conexion` es de hace menos de 30 s, el equipo está "EN LÍNEA"; si no, "DESCONECTADO".

Sobreconsumo. Si la potencia total supera 2500 W, el ESP32 abre los 3 relés en el momento (hay un solo sensor y no se sabe qué toma causó el exceso), escribe `rele_1, rele_2 y rele_3 = 0` y `estado/sobreconsumo = true`, y no vuelve a encender solo. Cuando el usuario enciende cualquier toma desde el panel (con confirmación), el panel pone `estado/sobreconsumo = false` y el equipo vuelve a obedecer.

Error de sensor. Si el PZEM-004T no responde, el ESP32 no inventa valores: marca `estado/error_sensor = true` y no actualiza `lectura_actual`.

Permisos en el panel.

Quién	Qué puede hacer
Visitante sin cuenta	Ver la presentación
Usuario	Vincular equipos y ver y controlar solo los suyos
Administrador	Además, ver todos los equipos, dar el rol de administrador y generar contraseñas temporales

Reglas de Firebase (`scripts/firebase-reglas.json`): el ESP32 entra con una cuenta de Firebase Authentication y solo puede escribir `info` (sin cambiar dueño ni nombre), el código (crearlo, no cambiarlo), `lectura_actual`, `estado`, `reles` e `historial`. `usuarios` y `eventos` quedan cerrados, igual que los nombres de las tomas: solo los maneja el panel con la clave secreta.

Valores únicos para todo el proyecto: umbral 2500 W, consumo fantasma de 0,5 a 15 W, factor de CO_2 0,283 kg/kWh (Inventario Nacional de GEI, dato 2022) y tarifa según una factura real.

Datos simulados

`scripts/simulador.php` se comporta como un ESP32 completo: se presenta en Firebase con su código, escribe con la misma forma de datos y respeta los 3 relés y el corte (cada toma simula un artefacto distinto). Se pueden correr varios a la vez para probar una cuenta con más de un equipo.

```
php scripts/simulador.php                          equipo EcoSmart_SIM001
php scripts/simulador.php --equipo=SIM002          otro equipo
php scripts/simulador.php --modo=sobreconsumo      provoca el corte
php scripts/simulador.php --modo=fantasma          aparatos en espera
php scripts/simulador.php --modo=desconectado      deja de escribir
```

Al arrancar muestra el identificador, el código y el link del QR para vincularlo desde el panel.

Reparto del trabajo y pendientes

Parte	Responsable	Qué hace
Firmware ESP32	Harry	ID por MAC, código, publicación en Firebase, 3 relés (uno por toma) y corte por sobreconsumo
Panel PHP	Dylan	Presentación, cuentas, vinculación, panel por equipo, actividad y administración
Simulador y reglas	Dylan	ESP32 falso y reglas de seguridad de Firebase

- [ ] Aprobar esta versión del contrato entre todos
- [ ] Compilar el firmware nuevo y probarlo con un equipo real
- [ ] Imprimir etiquetas con ID, código y QR para cada equipo
- [ ] Cargar en el `.env` el precio del kWh de una factura real
- [ ] Decidir si se mantiene Google Sheets
- [ ] Mostrar ID y código en la pantalla OLED (US-10)
- [ ] Guardar el historial en la Flash cuando no hay internet (US-11 y US-12)